

Rapport sur le mémoire de thèse intitulé
*Programmation logique pour les preuves interactives :
inférence des classes de type et analyse du déterminisme*
présenté par Monsieur DAVIDE FISSORE

Rapporteuse : Catherine Dubois, Professeure des Universités, ENSIIE, Samovar

Contexte de la thèse et résumé des contributions

La programmation logique et plus précisément le langage logique Elpi (un dialecte de λ -Prolog), offre un cadre bien adapté à la mise en œuvre de mécanismes d'automatisation dans l'assistant à la preuve Rocq et potentiellement d'autres outils de preuve, grâce notamment à ses mécanismes de recherche et d'unification. Rocq-Elpi, le plugin qui intègre Elpi à Rocq, ouvre ainsi la possibilité d'écrire en programmation logique des mécanismes d'automatisation habituellement enfouis dans le noyau de l'assistant, comme par exemple, la résolution des classes de type. Exploiter pleinement cette possibilité suppose cependant de résoudre deux difficultés liées à la combinaison d'Elpi et de Rocq : la représentation HOAS (*Higher Order Abstract Syntax*) des termes Rocq dans Elpi, qui fragilise l'unification, et l'absence de garanties sur le comportement fonctionnel des prédicats Elpi.

La thèse de Davide Fissore apporte des réponses à ces difficultés à travers quatre contributions : le développement, via Rocq-Elpi, d'un moteur de résolution de classes de type, alternatif à celui de Rocq ; une solution au problème de $\beta\eta$ -unification induit par l'encodage HOAS ; un analyseur statique du déterminisme pour Elpi ; et enfin une preuve formelle en Rocq de la correction de cet analyseur, pour le fragment du premier ordre.

Résumé et analyse du manuscrit

Le manuscrit, écrit en anglais, compte environ 130 pages et comprend sept chapitres et une bibliographie abondante. Le manuscrit est accompagné d'un artéfact librement accessible contenant le code Rocq correspondant au chapitre 6.

Le premier chapitre, intitulé « *Introduction* », constitue l'introduction de la thèse. Il donne le contexte, les motivations, et les contributions ainsi que l'organisation du manuscrit.

Le chapitre 2, « *Context : languages and tools* », introduit les bases nécessaires à la compréhension des contributions présentées dans les chapitres suivants. Il fait émerger naturellement les problèmes scientifiques qui motivent la suite du travail. Davide Fissore présente d'abord la notion d'élaboration dans un assistant à la preuve via l'exemple de *system F* et de l'inférence des paramètres de type puis il introduit les classes de types de Rocq ainsi que leur résolution. Vient ensuite la présentation du langage Elpi, qui définit formellement sa syntaxe abstraite et sa sémantique opérationnelle inspirée de celle proposée par Enrico Tassi dans son HDR (sans les *Constraint Handling Rules*, qui ne sont pas traitées par la suite). Le chapitre met l'accent sur les

traits principaux du langage Elpi, à savoir le cut - construction emblématique de Prolog et au centre de la thèse, pour contrôler le backtracking -, l'ordre supérieur, les clauses dynamiques et enfin la méta-programmation. Le chapitre se termine par la présentation de Rocq-Elpi, le plugin Elpi pour Rocq, qui permet l'intégration des deux langages. Les termes Rocq, représentés dans Elpi à l'aide d'une représentation HOAS, peuvent être manipulés directement par Elpi qui sert alors de méta-langage. Le document expose quelques APIs de Rocq utilisables dans Elpi et montre également comment écrire des tactiques et commandes à l'aide de Rocq-Elpi.

Le chapitre 3, intitulé « *Architecture of a Type-Class Solver in Rocq-Elpi* », concerne la première contribution de la thèse, à savoir le développement et l'évaluation d'un moteur de résolution de classes de types pour Rocq, implémenté en utilisant le langage Elpi et Rocq-Elpi. L'objectif est de proposer une alternative à la résolution de Rocq, ce qui paraît naturel car ce mécanisme est proche de la résolution de Elpi.

La présentation est progressive. Le chapitre commence par un compilateur basique, puis montre ses limites (mauvaise gestion des contextes locaux, des hypothèses d'ordre supérieur et des modes). Ce compilateur est ensuite raffiné pour prendre en compte les modes des paramètres des classes de type, les quantifications dépendantes, les positions positives ou négatives des termes et les priorités des règles. Les règles de résolution ad hoc peuvent être écrites en Elpi et ajoutées facilement dans la base de règles de Elpi. Ceci permet d'utiliser toute la puissance d'expression de Elpi, cependant au prix d'une maîtrise de ce langage par l'utilisateur de Rocq.

Davide Fissore a évalué son approche sur des bibliothèques existantes, en particulier TLC et STDPP qui utilisent largement les classes de type, et décrit quelques expériences comparant les performances du solveur Elpi avec celles du solveur de Rocq sur une instance relative à la composition de deux fonctions injectives (la classe de type étant celle qui spécifie la notion de fonction injective) en faisant varier la taille du problème à résoudre. La décomposition du temps de résolution pour Elpi (résolution des classes de types, traduction des termes Rocq en Elpi, communication entre Elpi et Rocq, et vérification de la preuve finale) met en évidence que le problème d'efficacité relève du typage du terme de preuve. Cette étude expérimentale est très intéressante et bien pensée. Elle est, cependant, limitée à une seule famille de problèmes construits artificiellement, il aurait été pertinent de l'étendre à d'autres types de problèmes issus de développements Rocq. L'utilisation de l'approche sur les bibliothèques STDPP et TLC a également permis d'identifier d'autres problèmes, en particulier un écart entre les modes de Rocq (utilisés pour contrôler la forme des arguments des classes de type) et ceux des prédicats de Elpi ainsi que la faiblesse de l'unification d'ordre supérieur côté Elpi ($\beta\eta$ -unification, δ -unification).

Le chapitre 4, intitulé « *Supporting $\beta\eta$ -unification in meta-programs* », apporte une solution élégante au problème de $\beta\eta$ -unification relevé au chapitre précédent. Le problème vient du fait que l'encodage HOAS des termes Coq dans Elpi génère des échecs lors de l'unification en Elpi. Plus généralement cela provient du fait que la théorie équationnelle de Rocq est plus riche que celle de Elpi. L'approche développée consiste à simplifier l'encodage Elpi des termes Rocq lorsque d'éventuels problèmes de $\beta\eta$ -unification sont détectés en remplaçant ces termes par des variables d'unification fraîches reliées aux termes originaux par des liens (*links*) qui encodent un problème d'unification suspendu. Ce nouvel encodage simplifie l'unification côté Elpi.

Dans le document, l'approche est généralisée à deux langages, $\mathcal{O}$ (le langage objet qui a une théorie riche) et $\mathcal{M}$ (le méta-langage qui a une théorie plus pauvre) et décrite formellement. La question générale concerne la simulation de l'exécution d'une séquence de problèmes d'unification entre termes de $\mathcal{O}$ dans le méta-langage $\mathcal{M}$. Un compilateur traduit les termes de $\mathcal{O}$ en un terme

en syntaxe abstraite d'ordre supérieur de $\mathcal{M}$ avec potentiellement des liens. Le compilateur est d'abord présenté pour traiter des problèmes du premier ordre puis étendu pour traiter les termes dits problématiques, de manière progressive : d'abord les problèmes liés à β , puis ceux liés à η , puis les variables apparaissant avec différentes arités (prise en compte des applications partielles de fonctions) et enfin les termes qui sortent du fragment de motif (ce dernier étant le terrain de jeu spécifique de Elpi). A chaque étape, des propriétés de fidélité sont démontrées, établissant que dans les domaines considérés, une exécution dans le langage objet $\mathcal{O}$ et son exécution compilée dans le méta-langage $\mathcal{M}$ restent en correspondance.

L'approche développée de manière abstraite est implémentée dans Elpi et Rocq-Elpi en utilisant son mécanisme de résolution de contraintes et les CHR pour réaliser la suspension et la notion de lien. L'implémentation est utilisée avec le solveur Elpi de classes de type du chapitre précédent. Ce dernier est plus rapide que celui de Rocq sur des problèmes de grande taille. L'évaluation expérimentale est très rapidement évoquée à la fin du chapitre. Une étude expérimentale plus avancée aurait été souhaitée, avec, par exemple, davantage de benchmarks et une mesure du coût de l'utilisation des liens. La détection des termes problématiques liés à η utilise une sur-approximation (maybe-eta) qui peut générer, dans certains cas, des liens inutiles. Il aurait été intéressant de mesurer le nombre de ces liens inutiles, par exemple sur les problèmes utilisés dans l'évaluation expérimentale du chapitre 4.

Le chapitre 5, « *Determinacy checking for Elpi* », ajoute à Elpi un analyseur de déterminisme permettant de vérifier qu'un prédicat est en fait une fonction, i.e. qu'une requête sur ce prédicat produit au plus une solution (il s'agit de la notion connue de déterminisme opérationnel). Cette problématique, classique en programmation logique, est envisagée, ici, pour (i) rendre Elpi plus accessible aux utilisateurs de Rocq, généralement plus familiers avec la programmation fonctionnelle qu'avec la programmation logique, et (ii) analyser statiquement qu'une classe de type se comporte de manière déterministe dans le cadre d'une résolution avec Elpi (donc pour compléter le solveur décrit au chapitre 3). L'analyse prend en compte les traits spécifiques à Elpi, à savoir les programmes d'ordre supérieur, dynamiques et les meta-programmes.

L'analyse repose sur la spécification par l'utilisateur d'une signature de déterminisme permettant d'indiquer qu'un prédicat doit se comporter comme une fonction (func ou pred dans la syntaxe concrète). Cette signature complète les modes associés aux paramètres (input ou output). Il est également possible de préciser pour un prédicat d'ordre supérieur qu'un paramètre doit être une fonction. Une relation d'ordre est définie sur les signatures.

L'algorithme vérifie que les règles d'un prédicat sont exclusives (en prenant en compte l'ordre et les priorités de celles-ci), que les prémisses de chaque règle utilise les prédicats conformément à leur signature et que la signature du prédicat est bien réalisée. L'algorithme est formalisé sous la forme d'un ensemble de règles d'inférence. Une particularité notable est que lors d'un appel non conforme à un prédicat fonctionnel (non respect des modes par exemple), ce dernier est déclassé en relation et l'analyse continue. L'algorithme ne prend pas en compte la forme des données et des résultats, ce qui limite quelque peu l'analyse d'exclusion mutuelle des règles d'un prédicat. Les signatures sont ajoutées manuellement par les utilisateurs lorsqu'ils définissent de nouveaux prédicats. Serait-il possible d'inférer les signatures possibles ?

Un théorème de correction (*soundness*) de l'algorithme par rapport à la sémantique opérationnelle d'Elpi (décrite au chapitre 2) est énoncé. La preuve formelle en Rocq de ce théorème restreint au premier ordre constitue la contribution décrite au chapitre suivant.

Davide Fissore a développé l'analyseur en OCaml en utilisant des arbres de discrimination pour

vérifier l'exclusion mutuelle et l'a intégré à Elpi (version 3.0). Il a été utilisé sur les bibliothèques Rocq-Elpi, Hierarchy Builder, Algebra Tactics, Trocq et BRiCk. Le résultat le plus impressionnant et convainquant concerne l'analyse de Rocq-Elpi : après insertion des signatures de déterminisme, 78 bugs ont été détectés et ont nécessité l'ajout de cut. 13 autres cut ont été ajoutés en raison des limites de la vérification de l'exclusion mutuelle qui n'est pas assez puissante pour démontrer l'exclusion de certaines règles. Il serait pertinent de compléter l'étude en détaillant combien, parmi ces bugs, donnaient effectivement plusieurs solutions, combien déclenchaient des recherches inutiles et en mesurant l'effet sur les performances. L'impact de l'utilisation de l'analyseur pour vérifier statiquement qu'une classe de type se comporte de manière déterministe lors de la résolution en Elpi ne me paraît pas clair et mériterait une discussion plus approfondie, compte tenu de la place de cette application dans la thèse. Enfin il serait intéressant d'avoir le retour des utilisateurs externes afin de mesurer si l'ajout des signatures et de l'analyseur facilite leur utilisation d'Elpi.

Le chapitre 6, « *A Mechanized First-Order Static Determinacy Checker* », présente la preuve formelle de la correction de l'analyseur de déterminisme introduit au chapitre précédent, en se limitant au fragment du premier ordre de Elpi. Dans cette analyse, le cut est central. Or, dans une sémantique opérationnelle classique comme celle donnée au chapitre 2, appelée ici sémantique par piles, il est difficile de caractériser précisément les alternatives supprimées par un cut. Les alternatives sont mélangées dans la structure de la pile. Davide Fissore introduit alors une seconde sémantique fondée sur des arbres Et-Ou (très similaires aux arbres de résolution) dans lesquels la structure de l'espace de recherche et l'effet du cut sont explicitement représentés. Dans cette sémantique, l'exécution d'une requête consiste à construire incrémentalement l'arbre Et-Ou correspondant et la rencontre d'un cut élague ce dernier. Le développement formel Rocq (qui utilise également Rocq-Elpi) comprend la définition de la syntaxe abstraite du langage et des arbres Et-Ou, la définition des deux sémantiques et la preuve de leur équivalence, la définition de l'analyseur statique de déterminisme, la preuve de *soundness* de ce dernier par rapport à la sémantique par arbres Et-Ou dont découle la preuve de *soundness* par rapport à la sémantique par piles.

La formalisation est très complète et va jusqu'à la formalisation de l'algorithme d'unification de Martelli-Montanari. Ce dernier est adapté de manière à prendre en compte un ensemble de variables dites gelées pour simuler le mécanisme de Elpi qui filtre les inputs et unifie les outputs d'un prédicat. La preuve de l'équivalence des deux sémantiques est très élégante et originale. La notion de d'arbre valide permet de définir précisément les arbres qui peuvent être obtenus pendant l'exécution d'une requête et ainsi mettre en relation les deux états de calcul, i.e. l'arbre Et-Ou et la pile d'alternatives. La lecture simultanée du code Rocq aurait pu être facilitée par l'ajout d'un guide indiquant où trouver les différents éléments définis dans le chapitre.

Le dernier chapitre, « *Conclusion and Perspectives* », fait le bilan des contributions et présente quelques perspectives, en particulier l'utilisation des liens (cf chapitre 4) pour traiter les structures canoniques et la poursuite de la formalisation en Rocq de la sémantique du langage Elpi, en prenant en compte toutes ses facettes.

Evaluation globale

Le manuscrit est bien structuré et bien écrit. Il comporte de nombreux exemples qui permettent d'appréhender aisément problèmes et solutions. Les formalisations mathématiques sont rigoureuses et claires. Le document se lit donc très agréablement.

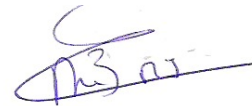
Le chapitre 2 fait émerger naturellement les problèmes scientifiques qui motivent la suite du

travail. Les contributions sont cohérentes et sont amenées naturellement, chapitre après chapitre. Je voudrais également souligner que les chapitres 3 à 6 contiennent tous une partie qui positionne la contribution correspondante par rapport aux travaux existants. Les contributions sont formellement présentées et justifiées, voire mécanisées dans un assistant à la preuve. Elles sont également implémentées et intégrées à Elpi et Rocq-Elpi. Leurs limites sont également discutées. Je regrette néanmoins que les tests ou résultats expérimentaux ne soient pas un peu plus détaillés dans le manuscrit, voire un peu plus avancés.

Les contributions, conséquentes et élégantes, combinent de manière très convaincante développement logiciel, résultats théoriques, évaluation expérimentale et formalisation dans un assistant à la preuve. Elles ont été publiées, pour la plupart, dans une conférence spécialisée dans la programmation logique et déclarative, reconnue par la communauté.

Recommandation

La qualité du manuscrit présenté ainsi que les résultats obtenus et leur pertinence me conduisent à donner **un avis très favorable** à la soutenance de la thèse de Davide Fissore pour l'obtention du titre de Docteur en Informatique de l'université Côte d'Azur.



Évry-Courcouronnes, le 29 août 2026
Catherine DUBOIS